

- Description

This module manages a smart parking lot system, ensuring that:

1. **Parking space availability is continuously monitored to guide drivers to free spots efficiently.**
2. **If the parking lot is full, incoming vehicles are added to a waiting queue and notified when spaces become available.**
3. **If multiple spots are available, the system distributes incoming vehicles evenly to balance congestion.**
4. **A log of all vehicle entries, exits, and waiting list updates is maintained for parking optimization.**

This system helps reduce congestion, improve parking efficiency, and optimize space utilization in urban areas.

- Specification

module SmartParkingLotManagement

var availableSpaces, totalSpaces: integer

var waitlist: sequence of string

var log: sequence of string

process parkVehicle(vehicle: string)

functional scenario form:

availableSpaces > 0 && availableSpaces = availableSpaces - 1, log = log $\oplus$ (vehicle + " parked | Remaining: " + availableSpaces + "/" + totalSpaces), Vehicle parked successfully.

||

availableSpaces = 0 && waitlist = waitlist $\oplus$ vehicle, log = log $\oplus$ (vehicle + " added to waitlist | No spaces available."), Vehicle added to waitlist.

end process

process leaveParking(vehicle: string)

functional scenario form:

(size(waitlist) > 0) && let nextVehicle = head(waitlist) in waitlist = tail(waitlist), log = log $\oplus$ (vehicle + " left | Assigned to " + nextVehicle), Parking assigned to next vehicle in waitlist.

OR

(size(waitlist) = 0) && availableSpaces = min(totalSpaces, availableSpaces + 1), log = log $\oplus$ (vehicle + " left | Available: " + availableSpaces + "/" + totalSpaces), Parking space returned to availability.

end process

end module

- Java Code

```
package code;
import java.util.*;
```

```

class ParkingLot {
    private int totalSpaces;
    private int availableSpaces;
    private Queue<String> waitlist;
    private List<String> log;
    public ParkingLot(int totalSpaces) {
        this.totalSpaces = totalSpaces;
        this.availableSpaces = totalSpaces;
        this.waitlist = new LinkedList<>();
        this.log = new ArrayList<>();
    }
    public void parkVehicle(String vehicle) {
        if (availableSpaces > 0) {
            availableSpaces--;
            System.out.printf("%s parked | Available Spaces: %d/%d\n",
vehicle, availableSpaces, totalSpaces);
            log.add(String.format("%s parked | Remaining: %d/%d", vehic
le, availableSpaces, totalSpaces));
        } else {
            waitlist.add(vehicle);
            System.out.printf("No spaces available. %s added to the wai
ting list.\n", vehicle);
            log.add(String.format("%s added to waitlist | No spaces ava
ilable", vehicle));
        }
    }
    public void leaveParking(String vehicle) {
        if (!waitlist.isEmpty()) {
            String nextVehicle = waitlist.poll();
            System.out.printf("%s left. %s can now park.\n", vehicle, n
extVehicle);
            log.add(String.format("%s left | Assigned to %s", vehicle,
nextVehicle));
        } else {
            availableSpaces = Math.min(totalSpaces, availableSpaces + 1
);
            System.out.printf("%s left | Available Spaces: %d/%d\n", ve
hicle, availableSpaces, totalSpaces);
            log.add(String.format("%s left | Available: %d/%d", vehicle
, availableSpaces, totalSpaces));
        }
    }
    public void printLog() {
        System.out.println("\nParking Lot Log:");
    }
}

```

```

        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

public class c60_SmartParkingSystem {
    public static void main(String[] args) {
        ParkingLot parkingLot = new ParkingLot(3);

        // testcase-VT:
        // parkingLot.parkVehicle("Car A");
        // parkingLot.parkVehicle("Car B");
        // parkingLot.parkVehicle("Car C");
        // parkingLot.parkVehicle("Car D");
        // parkingLot.leaveParking("Car B");
        // parkingLot.leaveParking("Car A");
        // parkingLot.printLog();

        // testcase-FT:
        // parkingLot.parkVehicle("mz");
        // parkingLot.parkVehicle("m0");
        // parkingLot.parkVehicle("m`");
        // parkingLot.parkVehicle("mo");
        // parkingLot.leaveParking("m/");
        // parkingLot.leaveParking("mm");
        // parkingLot.printLog();

        // testcase-Z3:
        // parkingLot.parkVehicle("Car A");
        // parkingLot.parkVehicle("Car B");
        // parkingLot.parkVehicle("Car C");
        // parkingLot.parkVehicle("Car D");
        // parkingLot.leaveParking("Car B");
        // parkingLot.leaveParking("Car A");
        // parkingLot.printLog();

        // testcase-UVT:
        parkingLot.parkVehicle("Car A");
        parkingLot.parkVehicle("Car B");
        parkingLot.parkVehicle("Car C");
        parkingLot.parkVehicle("Car D");
        parkingLot.leaveParking("Car B");
        parkingLot.leaveParking("Car A");
        parkingLot.printLog();
    }
}

```

```

    }
}

```

- Coverage report

1. Test case number: 6
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c60_SmartParkingSystem.java	100.0 %	6	0	6
> c60_SmartParkingSystem		0	0	0
> ParkingLot	100.0 %	6	0	6

3. Vibration testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	244	20,443	20,687
▼ src	1.2 %	244	20,443	20,687
▼ code	1.2 %	244	20,443	20,687
▼ c60_SmartParkingSystem.java	98.8 %	244	3	247
> c60_SmartParkingSystem	89.7 %	26	3	29
> ParkingLot	100.0 %	218	0	218

4. Fuzz testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c60_SmartParkingSystem.java	100.0 %	6	0	6
> c60_SmartParkingSystem		0	0	0
> ParkingLot	100.0 %	6	0	6

5. Fuzz testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	244	20,443	20,687
▼ src	1.2 %	244	20,443	20,687
▼ code	1.2 %	244	20,443	20,687
▼ c60_SmartParkingSystem.java	98.8 %	244	3	247
> c60_SmartParkingSystem	89.7 %	26	3	29
> ParkingLot	100.0 %	218	0	218

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c60_SmartParkingSystem.java	100.0 %	6	0	6
> c60_SmartParkingSystem		0	0	0
> ParkingLot	100.0 %	6	0	6

7. Z3 instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	244	20,443	20,687
▼ src	1.2 %	244	20,443	20,687
▼ code	1.2 %	244	20,443	20,687
▼ c60_SmartParkingSystem.java	98.8 %	244	3	247
> c60_SmartParkingSystem	89.7 %	26	3	29
> ParkingLot	100.0 %	218	0	218

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c60_SmartParkingSystem.java	100.0 %	6	0	6
> c60_SmartParkingSystem		0	0	0
> ParkingLot	100.0 %	6	0	6

9. Unimproved V-method instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	244	20,443	20,687
▼ src	1.2 %	244	20,443	20,687
▼ code	1.2 %	244	20,443	20,687
▼ c60_SmartParkingSystem.java	98.8 %	244	3	247
> c60_SmartParkingSystem	89.7 %	26	3	29
> ParkingLot	100.0 %	218	0	218